

Git and Engineering Practices

Contents

How Git works

1. [The object model](#)
2. [Refs, branches and HEAD](#)
3. [The three areas](#)
4. [Remotes, fetch and pull](#)

Everyday operations

5. [Merge and rebase](#)
6. [Rewriting history safely](#)
7. [Undoing things](#)
8. [Recovering with reflog](#)
9. [Stash and cherry-pick](#)
10. [Resolving conflicts](#)
11. [Finding a bad commit with bisect](#)

Working with others

12. [Branching strategies](#)
13. [Commit hygiene](#)
14. [Pull requests](#)
15. [Code review](#)
16. [Branch protection and enforcement](#)
17. [Shipping incomplete work](#)
18. [Monorepo and polyrepo](#)

Releasing

19. [Semantic versioning](#)
20. [Tags and release notes](#)

Reference

21. [Interview questions](#)
22. [Common mistakes](#)

1. The object model

Almost every confusing thing about Git stops being confusing once the storage model is clear, so it is worth the two minutes even though interviews rarely ask for it directly.

Git is a **content-addressed object store**. Every object is stored under a hash of its own contents, which means identical content is stored once and any change produces a different address.

Object	Holds
Blob	File contents. No name, no path, no permissions
Tree	A directory listing: names, modes, and the hash of the blob or tree each entry points at
Commit	One tree hash, zero or more parent commit hashes, author, committer, timestamps, message
Tag	An annotated tag: a named pointer at an object, with its own message and author

```
commit a1b2c3
  tree -----> tree 9f8e7d
  parent 4d5e6f    |- README.md ----> blob 1a2b3c
  author ...      |- src/ -----> tree 7c8d9e
  message ...                |- Order.java --> blob 5f6a7b
```

Four consequences worth being able to state:

- **A commit is a snapshot, not a difference.** It names a complete tree. Diffs are computed when we ask for them, by comparing two trees. This is why `git log -p` is doing work rather than reading something stored.
- **The parent links make history a directed acyclic graph.** A merge commit is simply a commit with two parents, which is why merges need no special machinery.
- **The commit hash covers the parents and the metadata,** not only the tree. So changing anything about a commit, including its message or its parent, produces a different commit with a different hash. That single fact is the whole of section 6.
- **Identical content maps to the same object.** A file unchanged across a thousand commits is one blob referenced a thousand times, rather than a thousand copies. How those objects are physically stored is a separate matter, since Git also delta-compresses them into packfiles.

Objects are addressed by SHA-1 today, with SHA-256 available in newer versions and not yet the common default. The distinction rarely matters in practice, and knowing the transition exists is enough.

2. Refs, branches and HEAD

A **ref** is a name that identifies an object, most often a commit. A **branch** is a ref pointing at a commit; an annotated tag is a ref pointing at a tag object, which in turn points at the commit. That is the entire mechanism, and it is far simpler than the vocabulary suggests.

```
cat .git/refs/heads/main      # a single line: the commit hash
```

A **branch is a movable pointer to one commit**. Creating one writes a lightweight reference holding that commit's identifier, which is why branching is effectively instant regardless of repository size, and why the word "branch" is misleading if it suggests a copy of anything.

HEAD is where we are now. Normally it is a **symbolic ref**, meaning it points at a branch rather than at a commit:

```
cat .git/HEAD                # ref: refs/heads/main
```

Committing moves the branch that HEAD points at, and HEAD follows because it is pointing at the name rather than the value.

Detached HEAD is when HEAD points straight at a commit instead of at a branch. It happens on `git checkout <hash>` or on checking out a tag. Nothing is broken and nothing is lost; the only real risk is that new commits made there are not on any branch, so once we leave, only the reflog knows where they were. The fix is to name them before leaving:

```
git switch -c experiment      # give the current position a branch name
```

The ref types worth distinguishing:

Ref	Meaning
<code>refs/heads/main</code>	A local branch
<code>refs/remotes/origin/main</code>	Our last observed position of the remote branch, updated by fetch. Not live
<code>refs/tags/v1.4.0</code>	A tag, which does not move
<code>HEAD</code>	The current position, usually symbolic
<code>ORIG_HEAD</code>	Where HEAD was before the last operation that moved it substantially

That second row explains a common confusion. `origin/main` is a cached observation, not a question asked of the server. It is stale until `git fetch`, which is why `git fetch` followed by `git log origin/main` shows something different from before.

Operation	What it touches
<code>git merge</code> or <code>git rebase</code>	Integrates a remote-tracking ref into the current branch
<code>git pull</code>	Fetch, then integrate. One command doing both
<code>git push</code>	Sends local commits and asks the server to move its branch

`git fetch` against `git pull` is one of the most common Git interview questions, and the answer is short:

`git pull` is `git fetch` followed by an integration step. Fetch is always safe, because it only updates our record of where the remote is. Pull changes the branch we are on, which is where conflicts and unexpected merge commits come from.

Which integration step `pull` performs follows the repository's pull configuration. Commonly that means a merge, and it can be configured to rebase instead or to allow only fast-forward updates. Being explicit avoids depending on whatever somebody set:

```
git pull --rebase           # replay our commits on top
git pull --ff-only         # refuse if integrating would need a merge
```

Fetching first and looking before integrating is a good habit, and it costs one extra command.

Ahead, behind and diverged describe the relationship between our branch and its remote-tracking ref, and the vocabulary is worth being precise about because it decides what to do next:

Behind	local A---B origin A---B---C	fast-forward is possible
Ahead	local A---B---C origin A---B	push is possible
Diverged	local A---B---C origin A---B---D	neither. Merge or rebase first

```
git status                 # says ahead, behind or diverged, after a fetch
git log --oneline --graph main origin/main
```

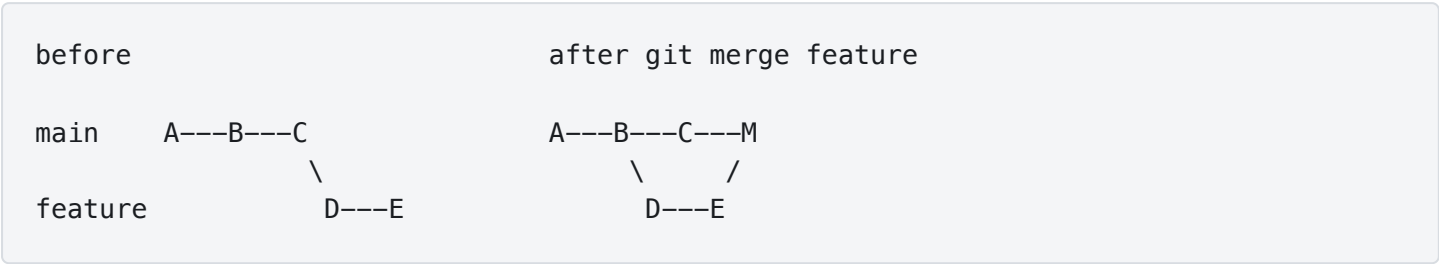
Diverged is the state worth recognising, because it is where a careless `git pull` creates a merge commit nobody wanted, and where a careless `git push --force` destroys commit `D`. The deliberate choices are to rebase our work onto `origin/main`, or to merge it, per section 5.

The caution from section 2 applies throughout: `status` compares against `origin/main`, which is our **last observation**. Before trusting any of the three words above, fetch.

5. Merge and rebase

The most frequently asked Git question, and the answer is a trade rather than a winner.

Merge creates a new commit with two parents, joining the histories without altering either.



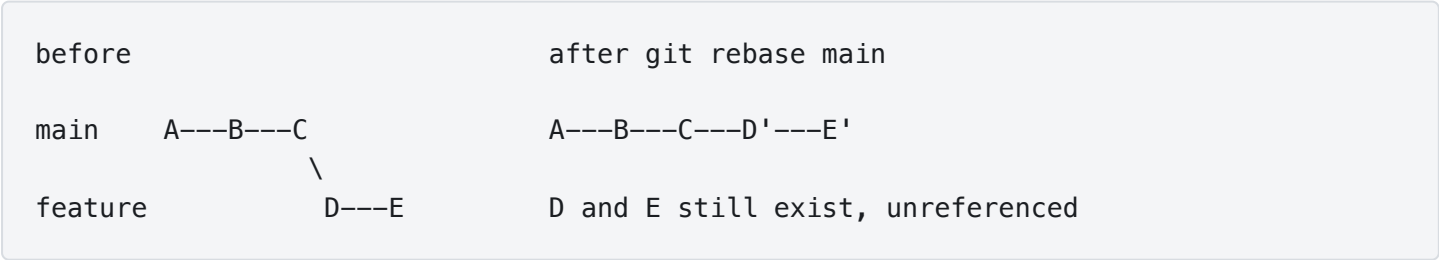
A **fast-forward** is the special case where the branch we are on is an ancestor of the target, so there is nothing to reconcile and Git just moves the pointer forward. No merge commit is created.

The two flags that control this are worth knowing as a pair:

Flag	Effect
<code>--no-ff</code>	Always create a merge commit, even when a fast-forward was possible. Keeps the shape of a feature visible in history
<code>--ff-only</code>	Allow only a fast-forward, or nothing to do at all. Refuse when integrating would require a merge

`--ff-only` is the more interesting of the two, because it turns an assumption into a check. Running `git pull --ff-only` says "integrate only if that needs no merge", so it succeeds when we are behind, succeeds with nothing to do when we are already up to date or ahead, and **stops when the histories have diverged** rather than quietly creating a merge commit nobody intended. That last case is a common source of accidental merge commits on a branch somebody meant to keep linear.

Rebase replays our commits on top of another branch, creating **new commits** with new hashes.



`D'` and `E'` contain the same changes as `D` and `E` and are different commits, because a commit hash covers its parents.

	Merge	Rebase
History	Records what actually happened, including the branching	A single line, easier to read and to bisect

	Merge	Rebase
Existing commits	Untouched	Replaced by new ones
Safe on shared branches	Yes	No, for the reasons in section 6
Conflicts	Resolved once, in the merge	Possibly once per replayed commit
Context preserved	The merge commit records the integration point	Lost, unless the message carries it

The convention that works for most teams: **rebase our own unpushed work to keep it tidy, merge to integrate**. So a feature branch is rebased onto the latest mainline before review, and the reviewed result is merged. That keeps individual history readable without rewriting anything anyone else has.

Worth knowing, because interviewers ask it as a follow-up: rebasing may replay a conflict once for every commit being moved, since each is applied in turn. `git rerere`, in section 10, is the setting that makes that bearable.

6. Rewriting history safely

Since a commit's hash covers its parents and its metadata, **changing anything about a commit replaces every commit after it**. Everything in this section follows from that one fact.

The operations:

```
git commit --amend           # replace the last commit
git rebase -i HEAD~5         # reorder, squash, edit, drop the last five
git rebase main              # replay this branch onto main
```

Interactive rebase is the general tool. It opens a list of commits with an action against each: `pick`, `reword` for the message alone, `squash` to fold into the previous commit keeping both messages, `fixup` to fold in and discard the message, `edit` to stop and amend, and `drop`.

The rule about shared history

Do not rewrite commits that other people already have. Their repository still references the originals, so when they next pull, Git tries to reconcile two histories containing the same changes under different hashes. The usual result is duplicated commits, a confusing merge, and someone restoring the old commits by accident.

The practical form: rewriting our own unpushed work is routine and expected. Rewriting anything pushed to a shared branch needs the people affected to know first, and rewriting a mainline branch is an incident rather than a technique.

When a force push is genuinely needed on our own branch:

```
git push --force-with-lease          # not --force
```

`--force-with-lease` refuses the push when the remote is not where we expected it to be, based on our remote-tracking ref, which is exactly the case where forcing would discard somebody else's work. Plain `--force` overwrites regardless. The difference costs nothing and is worth making a habit.

Rewriting history does not un-expose a secret

Removing a committed credential from history removes **references to it**. It does not remove the fact that the commit existed, was pushed, was fetched by everyone who pulled, was indexed by whatever scans the repository, and may sit in a fork, a build log, a mirror or a backup.

So the order is fixed: **rotate or revoke the credential first**, treating it as compromised from the moment it was pushed, and clean the history afterwards as tidying rather than as the fix. If it was never pushed, amending or rebasing it away is sufficient.

The durable fix is that the credential should never have been committable: secrets from a manager rather than a file, the file pattern in `.gitignore`, and a scanner in the pipeline that fails the build.

7. Undoing things

The area where the vocabulary genuinely is confusing, and interviewers probe it because the answer reveals whether someone understands the three areas from section 3.

Command	What it changes
<code>git reset --soft <commit></code>	Moves the branch pointer. Index and working tree untouched, so the changes are left staged
<code>git reset --mixed <commit></code>	Moves the pointer and resets the index. Changes remain in the working tree, unstaged. This is the default
<code>git reset --hard <commit></code>	Moves the pointer, resets the index and the working tree. Uncommitted work is discarded
<code>git revert <commit></code>	Makes a new commit that undoes an old one. Changes nothing that already exists
<code>git restore <path></code>	Restores a file in the working tree from the index
<code>git restore --staged <path></code>	Unstages a file, leaving the working tree alone

The distinction that matters most:

`reset` rewrites history. `revert` adds to it.

Which is why, for a commit already on a shared mainline, **revert is normally the right choice**. Reset would rewrite history other people already have, per section 6, while revert preserves it and leaves a truthful record that the change was made and then withdrawn, which is what an audit or an incident review needs. Deliberate history repair on a protected branch does happen, and it is a coordinated operation rather than a way of undoing a commit.

`git restore` and `git switch` were introduced to split the two unrelated jobs `git checkout` had accumulated: restoring files and moving between branches. `checkout` still does both, and the newer commands are worth using because the intent is visible in the command.

The one destructive command here is `git reset --hard`, and the thing to know is that it discards **uncommitted** work, which nothing can recover. Committed work is usually recoverable, through a ref or the reflog, for as long as the objects survive garbage collection.

8. Recovering with reflog

The single most useful command for the moment something has gone wrong, and one many people never learn.

The reflog is a local record of every position HEAD and each branch tip has held. It is not part of the repository's history, is not pushed or fetched, and exists only in our own clone.

```
git reflog                                # everywhere HEAD has been, most recent first
git reflog show main                      # everywhere main has pointed
```

```
a1b2c3 HEAD@{0}: reset: moving to HEAD~3
9f8e7d HEAD@{1}: commit: add discount tier lookup
4d5e6f HEAD@{2}: commit: extract pricing calculator
```

Recovering is then a matter of pointing something at the position we want:

```
git reset --hard HEAD@{1}                # back to before the reset
git branch recovered 9f8e7d              # or give the lost commits a name
```

What this covers: a bad reset, a deleted branch, a rebase that went wrong, an amend that discarded a message, commits made in detached HEAD. In each case the commits still exist as objects and have merely stopped being reachable by any name.

What it does not cover: anything never committed. A `git reset --hard` over uncommitted edits, or a `git checkout` over an unsaved file, is gone. This is the real argument for committing early and often on a private branch, since a messy commit can be tidied later by section 6 and a lost edit cannot be recovered at all.

Unreachable objects survive until garbage collection, and the default reflog retention is 90 days, or 30 for entries already unreachable. **That is a recovery window, not permanent storage.** Both settings are configurable, and a repository that has been aggressively pruned may have less. The usual failure is still not knowing to look rather than looking too late.

9. Stash and cherry-pick

Stash puts uncommitted changes aside and restores a clean working tree.

```
git stash push -m "half-done validation"
git stash push -u                # -u also stashes untracked files
git stash list
git stash pop                    # apply and remove
git stash apply                  # apply and keep
```

The flag worth remembering is `-u`. By default a stash leaves untracked files where they are, so a new file that was part of the work stays behind and confuses whatever we switched to.

Stash is convenient for interruptions and is worth treating as short-term. A stash has no branch, no message unless we supply one, and no visibility to anybody else, so work left in one for a week is work nobody can find. Committing on a branch is better for anything that will outlive the afternoon.

Worktrees answer the same question differently. Rather than putting work aside to free the one working tree, `git worktree` gives a second directory checked out to a different branch, sharing the same repository:

```
git worktree add ../hotfix main    # a second directory, on its own branch
git worktree list
git worktree remove ../hotfix
```

```
one repository
|- working tree: feature-a    our current work, untouched
+- working tree: hotfix       the interruption, in its own directory
```

Useful when switching context often, when a build takes long enough that losing its output hurts, or when reviewing somebody's branch without disturbing what is in progress. The cost is disk space and remembering to remove them.

Cherry-pick applies the changes from one commit as a new commit somewhere else.

```
git cherry-pick 9f8e7d
git cherry-pick 9f8e7d^..4d5e6f    # a range
```

Its legitimate use is narrow and worth stating precisely: **taking a specific fix to a release branch that must not receive everything else on the mainline**. That is a real and common need.

The mistake is using it as a general integration tool. A cherry-picked commit is a **new commit with different content lineage**, so the same change now exists twice in the history under two hashes. Git does not know they are related, so a later merge between those branches can conflict on a change that is logically already present on both. If we find ourselves cherry-picking routinely rather than merging, the branching strategy is producing branches that are not actually meant to converge.

10. Resolving conflicts

A conflict is not an error. It is Git declining to guess, which is the correct behaviour.

Why they happen: Git merges using a three-way comparison between the two branch tips and their **merge base**, the most recent commit both share. Where only one side changed a region, that side wins automatically. Where both changed the same region, Git stops and asks.

```
<<<<<<< HEAD
      return price.multiply(GOLD_RATE);
=====
      return price.multiply(discountFor(tier));
>>>>>>> feature/tiered-discounts
```

Above the `=====` is our side, below it is theirs. Both markers and the middle line must be gone before the file is valid; a conflict marker committed by accident is a common and embarrassing failure.

```
git status          # which files are unmerged
git diff            # the conflicting regions
git restore --ours <file>  # take our side wholesale (git checkout --ours
in older Git)
git restore --theirs <file> # take theirs wholesale
git add <file>        # mark resolved
git merge --abort     # or start over
```

The one thing to be careful about: during a **rebase**, "ours" and "theirs" are the reverse of what intuition suggests. The rebase is replaying our commits onto the other branch, so the branch being replayed onto is "ours" and the commit being applied is "theirs". Checking with `git status` before reaching for either flag is quicker than reasoning about it.

Two things that reduce the pain, both under our control:

- **Integrate often.** Conflict difficulty grows with how long two branches have diverged, so a branch rebased onto the mainline daily conflicts in small, comprehensible pieces. This is one of the strongest practical arguments for the branching strategies in section 12.

- **Turn on `rerere`**. It records how a conflict was resolved and reapplies that resolution automatically the next time the same conflict appears, which is what makes a long rebase tolerable.

```
git config --global rerere.enabled true
```

11. Finding a bad commit with bisect

Binary search over history, and the answer to "something broke somewhere in the last 200 commits and nobody knows where".

```
git bisect start
git bisect bad                # the current commit is broken
git bisect good v1.4.0        # this tag was fine
# Git checks out a commit halfway between. Test it, then:
git bisect good              # or: git bisect bad
# repeat until Git names the first bad commit
git bisect reset             # return to where we started
```

The cost is logarithmic, so 200 commits take about eight tests and 1000 take about ten. That is the whole reason it is worth knowing: the alternative is reading diffs until something looks suspicious, which scales badly and depends on the fault being visible.

Better still, automate it. Given any command that exits zero when the code is good and non-zero when it is bad, Git runs the whole search unattended:

```
git bisect run ./gradlew test --tests OrderPricingTest
```

What makes bisect work, and it is worth saying in an interview because it connects to commit hygiene: it needs most commits to build and run. Where a commit cannot be tested, `git bisect skip` excludes it and Git works around the gap, so the occasional broken commit is survivable. What is not survivable is a history where many of them are broken intermediate states, because each skip widens the range Git cannot narrow, and enough of them leave the search reporting a span of commits rather than one. That is a concrete payoff for the discipline in section 13, and a better argument for it than tidiness.

12. Branching strategies

Three well-known shapes, and the useful answer names what each fits rather than declaring one correct.

Git Flow. Long-lived `develop` and `main`, plus `feature/`, `release/` and `hotfix/` branches. Designed for software with versioned releases that are supported in parallel, which is exactly what it suits: installed software, mobile applications going through review, anything where several versions are live at once. Its cost is branches that live long enough to diverge substantially, and the merge pain that follows.

GitHub Flow. One long-lived branch. Short-lived feature branches off it, opened as a pull request, then merged. Lightweight, and it works particularly well where integration and deployment are relatively frequent, though nothing in it requires deploying on every merge.

Trunk-based development. Everyone commits to one branch, either directly or through branches measured in hours rather than days. Incomplete work ships behind a flag or an abstraction, per section 17.

	Suits	Costs
Git Flow	Multiple supported versions, scheduled releases	Long-lived branches, painful merges, slow feedback
GitHub Flow	Continuous deployment of a single version	Needs solid automated tests and quick review
Trunk-based	Frequent integration, larger teams, continuous delivery	Needs feature flags, discipline, and a reliable build

The principle underneath all three, and the part that transfers: **the longer a branch lives, the more expensive it is to integrate**, because divergence compounds. Every strategy above is a different answer to the same question of how to keep that cost down, and the research behind the DORA metrics in the cloud note points consistently at frequent integration rather than at any specific branch layout.

Worth being honest in an interview: a team's strategy is usually inherited rather than chosen, and the useful skill is working well within one and recognising when its costs have outgrown its benefits. Arguing for trunk-based development on a team that releases four supported versions a year is not the answer that lands.

13. Commit hygiene

An atomic commit contains one logical change and leaves the code working. That is the whole standard, and everything else follows.

Why it matters, beyond neatness, and these are the reasons to give when asked:

- **Review.** A reviewer can read one intention at a time rather than untangling three.
- **Bisect.** Section 11 only works if every commit builds.
- **Revert.** A single change can be withdrawn without taking unrelated work with it.
- **Blame.** `git blame` leads to a commit that explains one thing, rather than to "various fixes".

A message with a body has a conventional shape:

Short summary in the imperative, under about 50 characters

Explain why the change was needed and what approach it takes. The diff already shows what changed, so the body is for the reasoning that the diff cannot carry: what was tried, what was rejected, what constraint forced this shape.

Fixes PROJ-1234

The summary is written in the imperative, "add", "fix", "remove", because it completes the sentence "applying this commit will ...", which is also how Git's own generated messages read.

The body is where the value is. A diff shows what changed; nobody can reconstruct why. The commit that says "use a pessimistic lock here because the optimistic retry starved under contention" saves somebody a day, possibly us.

Conventional Commits is a machine-readable convention on top of that:

```
feat(pricing): apply tier discounts at checkout
fix(orders): stop double-charging on retried payments
docs(readme): document the local database setup
```

Types are `feat`, `fix`, `docs`, `refactor`, `test`, `chore` and others by agreement, with `!` or a `BREAKING CHANGE:` footer marking an incompatible change. Its payoff is automation: version numbers and changelogs can be generated from the log, per sections 17 and 18. Its cost is a small tax on every commit, so it earns its place where the automation is actually wired up and is ceremony where it is not.

Hooks enforce some of this locally. A `pre-commit` hook running the formatter and the linter catches problems before they reach review. Two cautions: hooks live in `.git/hooks`, which is not cloned, so a team needs a tool to install them; and a slow hook gets bypassed with `--no-verify`, which means the check has to be quick or it will simply be skipped.

14. Pull requests

A pull request should be small enough to review properly. The reason is not politeness. Review quality falls as size grows, and past a certain point reviewers stop finding defects and start approving, because the effort to review honestly has become larger than the effort to trust the author.

The failure is recognisable: a 40-file pull request collects "looks good to me" within the hour, and a 6-file one collects real questions. The larger one was not better.

Ways to keep them small, in rough order of usefulness:

- **Separate refactoring from behaviour change.** A commit that moves code and a commit that changes what it does are each readable; combined, neither is. This is the single most effective habit here.

- **Merge groundwork first.** Interfaces, configuration and scaffolding can go in ahead of the change that uses them.
- **Ship behind a flag.** Per section 17, incomplete work can be merged safely rather than accumulating on a branch.
- **Stack the changes.** Several dependent pull requests, each reviewable, rather than one that is not.

A description worth reading answers three things, and only the third is optional:

Why	The problem, and the ticket. What breaks without this
What	The approach, and what was rejected. This is what reviewers most often lack
How to check	What to run, what to look at, anything not covered by tests

The self-review before requesting one is worth the five minutes: reading our own diff in the review interface, rather than in the editor, reliably surfaces debug output, commented-out code, and the file that was not meant to be in the change.

15. Code review

Review has two jobs that pull in different directions: **finding defects**, and **spreading knowledge about the codebase**. A review optimised only for the first becomes an inspection nobody learns from; one optimised only for the second stops catching anything.

A useful default order for review attention, which roughly tracks how expensive each problem is to fix later. A security-sensitive change or a data migration reorders it, and knowing when to reorder it is the skill:

1. **Correctness.** Does it do what it claims? What about the empty case, the concurrent case, the failure of the thing it calls?
2. **Security.** Untrusted input, authorisation on the new path, secrets, what gets logged.
3. **Design.** Does this belong here? Is it the same shape as the code around it? Will the next change be easy or awkward?
4. **Tests.** Do they assert anything meaningful, and would they fail if the behaviour broke? The testing note goes into what that means.
5. **Readability.** Naming, structure, and whether the comments explain the reasoning rather than restating the code.

What not to spend review on: **anything a tool can decide**. Formatting, import order and line length belong to a formatter in the pipeline, and arguing them in review costs attention that the first three items needed.

How the comment is written changes what happens next, and this is a real skill rather than politeness:

Instead of	Write
"This is wrong."	"What happens here if <code>items</code> is empty? I think this throws."
"Use a map."	"A map keyed by tier would make this O(1) rather than scanning. Worth it here?"
"Bad naming."	" <code>data</code> is hard to follow three methods later. <code>pendingOrders</code> ?"

Two conventions that make reviews finish: **mark what is optional**, with a prefix such as `nit:`, so the author knows what blocks the merge; and **approve with comments** where the remaining points are minor, rather than holding the change for another round.

The receiving side matters too. A review comment is about the code, and a question about a decision is usually a question rather than an accusation. Where a reviewer has misunderstood, the code was probably unclear, and the useful response is often a comment in the source rather than a reply in the thread that nobody reads again.

16. Branch protection and enforcement

Review, hooks and CI are often discussed as one thing, and they are three layers with different strengths. The distinction is a fair interview question because it exposes whether someone has thought about what actually guarantees anything.

Layer	Strength
Local hooks	Fast feedback, before a commit exists. Bypassable with <code>--no-verify</code> , and not cloned with the repository
CI checks	Runs the real verification on a clean machine. Cannot be skipped locally, but on its own it only reports
Branch protection	Refuses the merge. Of these three, the layer that can enforce policy rather than only report

The relationship worth stating plainly: **hooks are a convenience, CI is verification, and branch protection is what enforces merge policy**. A pre-commit hook running the formatter saves a round trip through review, and it guarantees nothing, because it lives in `.git/hooks`, which is not cloned, and any developer can pass `--no-verify`. Treating it as a guarantee is the mistake.

These three are the common arrangement rather than the only mechanisms available. Server-side hooks, merge queues and organisational policy also enforce, and the point that transfers is the distinction itself: a check a developer can skip locally is not a control, whatever it happens to be called.

What protection on a mainline branch typically requires:

- **No direct pushes.** Changes arrive through a pull request, which is what makes everything else applicable.

- **Status checks must pass.** The build and the tests, specifically the ones that would catch the failure being guarded against.
- **A required number of approvals**, often with the rule that a new push dismisses stale approvals.
- **No force pushes and no deletion**, which is the mechanical enforcement of the rule in section 6.
- **Signed commits**, where the provenance of a change matters.
- **Up to date with the base branch** before merging, so the tested combination is the one that lands.

CODEOWNERS maps paths to the people or teams who must review changes to them:

```
/src/payments/    @org/payments-team
/infra/          @org/platform
*.sql            @org/data
```

Its value is that it routes automatically, so a change to the payment code cannot be reviewed only by people unfamiliar with it. Its cost is that it becomes a queue: a team named on a widely-touched path becomes a bottleneck, and paths that are too broad produce approvals given without real attention, which is worse than no rule.

The general principle, and the one that transfers to any tool: **put the check at the layer that matches the consequence**. Formatting belongs in a hook, because being wrong costs a round trip. A failing test belongs in a required status check, because being wrong ships a defect. Who may approve a change to the payment code belongs in protection, because being wrong is not recoverable by trying again.

17. Shipping incomplete work

The technique that makes short-lived branches possible, and the answer to the obvious objection to trunk-based development: how do we merge daily when the feature takes three weeks?

Separate deploying from releasing. Code can be in production without being reachable. Once that separation exists, the branch no longer has to survive until the feature is finished.

Feature flags. The mechanism, covered in the cloud note. Two things belong here: a flag is a branch in the code rather than in Git, so it carries its own cost in paths that must be tested; and every flag needs a named owner, a stated scope and an expected removal date, because "remove it later" without a date reliably becomes never, and a codebase where flags accumulate is one where no path is well understood.

Branch by abstraction. For a change too structural for a flag, such as replacing a dependency, introduce an abstraction over the existing behaviour first, then move implementations behind it:

```
// 1. Introduce the abstraction over what exists today.
public interface PaymentGateway {
    PaymentResult charge(Money amount, Card card);
}

// 2. The current behaviour becomes one implementation. Nothing else changes.
public class LegacyGateway implements PaymentGateway { ... }

// 3. The replacement is built alongside, merged, and not yet used.
public class StripeGateway implements PaymentGateway { ... }

// 4. Switch over, one caller at a time, then delete the old one.
```

Each step is a small, mergeable, reversible change, and at no point does the mainline hold something that does not work.

Expand and contract is the same idea for data, and the reason it appears here rather than only in the database note is that it is what makes a schema change compatible with short-lived branches. Add the new column, write to both, migrate the readers, then remove the old one. Each step is independently deployable, and no step requires the code and the schema to change at the same instant.

18. Monorepo and polyrepo

Monorepo: many projects in one repository. **Polyrepo:** one repository per service or library.

	Monorepo	Polyrepo
Cross-project change	One commit, one review, atomic	Coordinated changes across repositories, merged in order
Dependency versions	One version of everything, always current	Each consumer upgrades on its own schedule
Discoverability	Everything is searchable in one place	Needs tooling or knowledge to find things
Access control	Harder to restrict by area	Natural boundary per repository
Build	Needs tooling to build only what changed	Simple per repository
Scale	Requires investment as the repository grows	Scales without special tooling

Neither is correct in general, and the honest framing is that this is a trade between **coordination cost** and **isolation**.

A monorepo makes the cross-cutting change cheap and the build system expensive. The expense is specific: a build that rebuilds everything on every commit stops being usable long before the repository is large, so a monorepo needs **affected-project detection**, where the tool works out which projects a change actually touches and builds only those plus whatever depends on them.

```
Only service A changed
-> build and test A, and everything that depends on A
-> leave the other forty projects alone
```

A polyrepo makes each repository simple and the cross-cutting change expensive, because a change spanning four services becomes four pull requests that have to merge in a particular order, and a version bump that has to propagate.

The factor that usually decides it is **how often changes cross project boundaries**. Services that genuinely evolve independently do well in separate repositories. A set of services that are really one system, changing together most weeks, pays the polyrepo cost repeatedly.

Worth knowing: the two are not the same question as monolith against microservices. A monorepo can contain many independently deployed services, and a polyrepo can contain one monolith split for no reason. Repository layout is about how code is stored and changed; service boundaries are an architecture question, covered in the architecture note.

19. Semantic versioning

`MAJOR.MINOR.PATCH`, where each part has a defined meaning:

Part	Increment when
MAJOR	An incompatible change. Callers must do something to upgrade
MINOR	Functionality added, existing callers unaffected
PATCH	A fix, no interface change

So `2.4.1` to `2.5.0` should be a safe upgrade, and `2.5.0` to `3.0.0` should not be. That is the promise, and its value is that a consumer can decide whether to read the release notes based on the number alone.

Two honest limits, and offering them is what turns a definition into a discussion:

- **What counts as breaking is a judgement.** Fixing a bug breaks anyone who depended on the bug. Tightening validation breaks whoever was sending invalid input successfully. This is Hyrum's Law in practice: with enough consumers, every observable behaviour of a system becomes something somebody relies on, regardless of what was promised.
- **It describes the interface, not the risk.** A patch release can carry a serious performance regression while remaining entirely compatible, so the version number is not a substitute for reading what changed.

`0.x.y` means the public interface is not yet covered by the compatibility guarantees above, which is a real signal rather than a formality. It is not a licence to break things arbitrarily; it says consumers cannot rely on the version number alone to tell them whether an upgrade is safe. Pre-release and build metadata are appended as `1.4.0-rc.1` and `1.4.0+build.240`.

For an internal service behind an API rather than a published library, the versioning that matters is usually the API's, covered in the API note. Applying semantic versioning to a deployable that nobody depends on by version is ceremony.

20. Tags and release notes

A tag names a commit permanently. Unlike a branch it does not move, which is what makes it useful for marking what was released.

```
git tag -a v1.4.0 -m "Release 1.4.0"    # annotated: has author, date, message
git tag v1.4.0                          # lightweight: just a pointer
git push origin v1.4.0                  # tags are not pushed by default
```

Use annotated tags for releases. A lightweight tag is a bare pointer with no record of who made it or when, which is exactly the information wanted later. The `git push` line above is the one most often forgotten: tags do not travel with a normal push.

`git describe` then names any commit relative to the last tag, which is what build pipelines usually stamp into an artefact:

```
git describe --tags                      # v1.4.0-12-ga1b2c3
```

Read as: twelve commits after `v1.4.0`, at commit `a1b2c3`.

Release notes are written for the people affected, which is what separates them from a changelog generated out of the log. A list of commit summaries tells a consumer nothing about whether to upgrade. What they need is what changed for them, what they must do, and what to be careful about.

The structure that works: **breaking changes first**, with the migration step for each; then what was added; then what was fixed; then anything deprecated with a date. Anything requiring action from the reader goes above anything that does not.

Generating this from Conventional Commits, per section 13, works well for the additions and fixes and works poorly for breaking changes, because the migration instruction is the part that has to be written by a person. Automating the easy half is worthwhile as long as the half that matters is still written.

21. Interview questions

As in the other notes, each question has an **opening** of roughly thirty to forty seconds, whose last sentence points at the deeper material, and a **follow-up** for when the interviewer takes it.

"What is HEAD, and what is a detached HEAD?"

HEAD is where we are now. Normally it is a symbolic ref, meaning it points at a branch name rather than at a commit, which is why committing moves the branch and HEAD follows along. A detached HEAD is when it points straight at a commit instead, which happens on checking out a hash or a tag. Nothing is broken when that happens, and the name makes it sound far more alarming than it is.

If they go deeper:

The one real risk is that commits made while detached are not on any branch, so once we switch away, nothing references them and only the reflog knows where they were. The fix is to name the position before leaving, with `git switch -c` and a branch name. It is worth connecting this to the general model: a branch is just a movable pointer to one commit, HEAD is a pointer to a branch, and almost everything confusing about Git becomes simple once those two facts are in place. Detached HEAD is not a special mode, it is HEAD holding a commit identifier rather than a branch name.

"What is the difference between `git fetch` and `git pull`?"

`git pull` is `git fetch` followed by an integration step, so the difference is what they touch. Fetch only updates our remote-tracking refs, meaning our record of where the server's branches are, and it changes no local branch and no file on disk, which makes it always safe. Pull changes the branch we are on, which is where conflicts and unexpected merge commits come from. Which integration step pull performs is a matter of configuration.

If they go deeper:

It follows the repository's pull configuration, which commonly means a merge and can be set to rebase or to allow only fast-forwards, so I prefer to be explicit with `--rebase` or `--ff-only` rather than depend on whatever somebody configured. `--ff-only` is the useful one, because it turns an assumption into a check: it integrates only when that needs no merge, so it is fine when I am behind or already up to date, and it stops rather than quietly creating a merge commit when the histories have diverged. That connects to the thing people most often get wrong about remotes, which is reading `origin/main` as the live state of the server. It is our last observation of it, and it is stale until we fetch, so "am I ahead, behind or diverged" is only a meaningful question after a fetch.

"What is the difference between merge and rebase?"

Merge creates a new commit with two parents, joining the histories without changing anything that already exists. Rebase replays our commits on top of another branch, which creates new commits with new hashes, because a commit's hash covers its parents as well as its content. So the trade is that merge records what actually happened and leaves a messier graph, while rebase gives a single readable line of history at the cost of replacing commits. Which matters mostly depends on whether anyone else has those commits.

If they go deeper:

If they are only ours and unpushed, rebasing is routine. If they are on a shared branch, rebasing means everybody else's repository still references the originals, so their next pull tries to reconcile two histories holding the same changes under different hashes, and the usual result is duplicated commits and a confusing merge. The convention I follow is rebase our own work to keep it tidy, merge to integrate. One practical detail worth knowing is that a rebase can replay the same conflict once per commit being moved, since each is applied in turn, which is why turning on `rerere` is worth it: it records a resolution and reapplies it automatically.

"What happens if you rebase a branch that has already been pushed?"

The rebase replaces every commit with a new one, because a commit's hash covers its parents, so the branch on the server and the branch in my repository now hold the same changes under different hashes. Pushing it needs a force, and anybody who already pulled the old commits still references them. When they next pull, Git tries to reconcile the two histories and the usual result is duplicated commits and a merge nobody intended.

If they go deeper:

If it is my own branch and nobody else is on it, this is routine, and I would push with `--force-with-lease` rather than `--force`, because the lease refuses when the remote is not where I expected and that is exactly the case where forcing destroys someone's work. If other people are on the branch, the fix is to tell them first, and what they need to do is reset their local branch to the remote rather than pull. On a shared mainline I would not do it at all: that is deliberate history repair, a coordinated operation with everyone notified, not a technique for tidying commits. The version of this rule I actually follow is that rewriting unpushed work is free, and rewriting anything anybody else has costs a conversation.

"How do you undo a commit?"

It depends entirely on whether anyone else has it. If it is only local, `git reset` moves the branch pointer back, and the flag decides how much else it takes with it: `--soft` leaves everything staged, `--mixed` leaves the changes in the working tree unstaged, and `--hard` discards them. If the commit is already on a shared branch, the answer is `git revert`, which creates a new commit undoing the old one and changes nothing that already exists.

If they go deeper:

The distinction I would summarise as reset rewrites history and revert adds to it, which is why revert is normally the right choice on a shared mainline. It also leaves a truthful record that the change was made and then withdrawn, which is what an incident review actually wants. The one genuinely dangerous command in that list is `git reset --hard`, and specifically because it discards uncommitted work, which nothing recovers. Committed work is usually recoverable through the reflog for as long as the objects survive garbage collection, so on a private branch I would rather commit something messy and tidy it later with an interactive rebase than risk losing it.

"I have just deleted a branch with unmerged work. What now?"

The reflog, and the work is almost certainly fine. Deleting a branch removes a pointer, not the commits, so they still exist as objects and have only become unreachable by name. `git reflog` shows every position HEAD has held, so I find the commit at the tip of that branch and give it a name again with `git branch recovered <hash>`. It takes about thirty seconds once we know to look there.

If they go deeper:

The reflog is purely local, so it is not pushed or fetched and it only covers our own clone, and unreachable entries are kept for about thirty days by default, which I would describe as a recovery window rather than permanent storage since both the retention and the pruning are configurable. The same command covers a bad reset, a rebase that went wrong, an amend that lost a message, and commits made in a detached HEAD. What it cannot recover is anything never committed, which is the real reason to commit early on a private branch. A messy commit can always be tidied later; an edit destroyed by `reset --hard` is simply gone.

"You have just run `git reset --hard` and lost work. What now?"

It depends entirely on whether the work was committed, and those are two completely different answers. If the commits existed, they still exist as objects and have only become unreachable, so `git reflog` shows every position HEAD has held, and I find the one I want and point a branch at it with `git branch recovered <hash>`. If the work was never committed, it is gone. `reset --hard` overwrites the working tree, and there is no Git-side record of something that was never an object.

If they go deeper:

That asymmetry is the real lesson, and it is why I commit early and often on a private branch: a messy commit can always be tidied later with an interactive rebase, and an uncommitted edit destroyed by `reset --hard` cannot be recovered at all. It is worth being precise that the reflog is a recovery window rather than permanent storage, since unreachable entries are kept about thirty days by default and both the retention and the pruning are configurable. The other places worth trying before giving up on uncommitted work are outside Git entirely, so the editor's local history or the build tool's output directory, which have saved people more than once.

"How would you find which commit introduced a bug?"

`git bisect`, which is a binary search over history. I mark the current commit bad and a known good one, usually the last release tag, and Git checks out the midpoint for me to test. Each answer halves the range, so 200 commits take about eight tests rather than 200. If there is a command that fails on the bug, I would not do it by hand at all: `git bisect run` with that command searches unattended.

If they go deeper:

The thing that makes it work is that most commits have to build and run, which is the practical argument for atomic commits rather than tidiness. An untestable commit can be excluded with `git bisect skip` and Git works around it, so the occasional broken one is fine. What is not fine is a history where many of them are broken intermediate states, because every skip widens the range Git cannot narrow, and enough of them leave the search naming a span of commits rather than one. So commit discipline returns the effort the first time somebody needs this. If the history is not bisectable, the fallback is narrowing by hand with `git log` on the affected paths, which is a lot slower and depends on the fault being visible in a diff.

"What branching strategy do you use, and why?"

The one the team already uses, and I would rather talk about what each fits. Git Flow with long-lived develop and release branches suits software with several supported versions live at once. GitHub Flow is one long-lived branch with short feature branches and pull requests, which works well where integration and release are relatively frequent, continuous deployment being one example rather than a requirement. Trunk-based development goes further and keeps branches to hours, with incomplete work behind flags. The principle under all three is the one that transfers.

If they go deeper:

That principle is that the longer a branch lives, the more expensive integrating it becomes, because divergence compounds and conflicts arrive all at once at the end. Every one of those strategies is a different answer to keeping that cost down, and the DORA research points consistently at frequent integration rather than at any particular branch layout. What I would avoid in an interview is arguing for trunk-based development regardless of context, because on a team supporting four released versions it is the wrong answer. The useful skill is working well inside the strategy a team has and recognising when its costs have outgrown its benefits.

"What makes a good commit and a good pull request?"

For a commit, one logical change that leaves the code working, with a message whose body explains why rather than what, since the diff already shows what. For a pull request, small enough to review properly, and that one is not about politeness. Review quality falls as size grows, and past a point reviewers stop finding defects and start approving, because reviewing honestly has become more work than trusting the author. A forty-file change collects "looks good to me"; a six-file change collects real questions.

If they go deeper:

The most effective habit for keeping them small is separating refactoring from behaviour change, because a commit that moves code and a commit that changes what it does are each readable and the combination is neither. Beyond that, merging groundwork ahead of the change that uses it, shipping behind a flag so incomplete work does not accumulate on a branch, and stacking dependent pull requests. Atomic commits also pay off in three places besides review: bisect needs every commit to build, revert can withdraw one change without taking unrelated work, and blame leads to a commit that explains one thing rather than to "various fixes".

"How do you ship a feature that takes three weeks without a long-lived branch?"

By separating deploying from releasing, so code can be in production without being reachable. The usual mechanism is a feature flag, which turns the branch in Git into a branch in the code. For something too structural for a flag, such as replacing a dependency, branch by abstraction: introduce an interface over the existing behaviour, build the replacement behind it, switch callers over one at a time, then delete the old one. Every step is small, mergeable and reversible.

If they go deeper:

The equivalent for data is expand and contract: add the new column, write to both, migrate the readers, then drop the old one, so no step needs the code and the schema to change at the same instant. The cost I would be honest about is that a flag is a branch in the code, so it carries paths that have to be tested, and every flag needs an owner and an expected removal date rather than a vague intention, because "later" without a date becomes never. A codebase where flags accumulate is one where nobody knows which paths are live, which is a worse problem than the long-lived branch it avoided.

"Monorepo or polyrepo?"

It is a trade between coordination cost and isolation rather than a correct answer. A monorepo makes the cross-cutting change cheap, one commit and one review, and makes the build system expensive, because it needs affected-project detection to build only what changed and whatever depends on it, rather than rebuilding everything on every commit. A polyrepo keeps each repository simple and makes the cross-cutting change expensive, since a change spanning four services becomes four pull requests merged in order plus a version bump that has to propagate. The factor that usually decides it is how often changes actually cross project boundaries.

If they go deeper:

Services that genuinely evolve independently do well separately. A set of services that are really one system, changing together most weeks, pays the polyrepo cost over and over. The other thing I would separate is that this is not the same question as monolith against microservices: a monorepo can hold many independently deployed services, and a polyrepo can hold a monolith split for no particular reason. Repository layout is about how code is stored and changed; service boundaries are an architecture decision, and conflating the two is how teams end up with the costs of both.

"An API key was committed and pushed. What do you do?"

Rotate it first, and treat it as compromised from the moment it was pushed. That ordering is the whole answer, because rewriting history removes references to the secret and does not remove the fact that it was on the server, was fetched by everyone who pulled, and may now sit in a fork, a build log, a mirror or a backup. Cleaning the history is tidying up afterwards, not the fix. If it was never pushed, amending or rebasing it away genuinely is sufficient.

If they go deeper:

After rotating I would check whether the old credential was used in the window it was exposed, because if it was, this stops being an exposure to clean up and becomes an incident with a scope to establish. Then the history cleanup, which on a shared branch means coordinating with everyone, since it rewrites commits they have. The part I would want to talk about is preventing the next one, because this always recurs: secrets from a manager rather than a file, the file pattern in `.gitignore`, and a secret scanner as a required check in the pipeline so the push is refused rather than reviewed. A pre-commit hook helps too, and I would not present it as the control, because it is not cloned with the repository and anyone can pass `--no-verify`.

"What do you look for in a code review?"

Correctness first, so the empty case, the concurrent case, and what happens when the thing it calls fails. Then security, meaning untrusted input, authorisation on the new path, and what ends up in the logs. Then design: whether it belongs here, whether it matches the code around it, and whether the next change will be awkward. Then whether the tests would actually fail if the behaviour broke. Readability last, and formatting not at all, because that belongs to a tool.

If they go deeper:

I try to remember review has two jobs that pull against each other, finding defects and spreading knowledge, and a review optimised only for the first becomes an inspection nobody learns from. How the comment is written changes what happens next, so I would rather ask "what happens here if items is empty, I think this throws" than write "this is wrong", because the question gets a fix and the statement gets a defence. Marking which comments are optional, with a nit prefix, is what lets a review finish. And when a reviewer has misunderstood something, the code was probably unclear, so the useful response is often a comment in the source rather than a reply in a thread nobody reads again.

22. Common mistakes

- Rewriting history that other people already have, which leaves them reconciling two histories holding the same changes under different hashes.
- Using `--force` where `--force-with-lease` would have refused and saved somebody's work.
- Reaching for `reset` to undo a commit already on a shared branch, where `revert` is the only correct answer.
- Running `git reset --hard` with uncommitted work in the tree, which is the one thing the reflog cannot bring back.
- Not knowing the reflog exists, and rebuilding lost work by hand instead.
- Reading `origin/main` as the current state of the remote, when it is our last observation of it and stale until `git fetch`.
- Running `git pull` on a diverged branch and getting a merge commit nobody intended, where `--ff-only` would have stopped and asked.
- Rewriting history to remove a pushed secret and treating that as the fix, when the credential needed rotating first and the cleanup was only tidying.
- Treating a pre-commit hook as a guarantee, when it is not cloned with the repository and `--no-verify` skips it. Enforcement lives in branch protection.
- Naming a team in CODEOWNERS for a path touched by everything, which turns a routing rule into a queue and produces approvals given without attention.
- Committing a conflict marker, because the file was staged without being read.
- Cherry-picking as a routine integration tool, so the same change exists twice under two hashes and a later merge conflicts on something already present on both sides.

- Leaving work in a stash for a week, where it has no branch, no context and no visibility to anyone else.
- Letting a branch live for weeks, so the conflicts all arrive at once at the end.
- Mixing refactoring and behaviour change in one commit, so neither is reviewable and neither can be reverted alone.
- Writing a commit message that restates the diff instead of recording why the change was needed.
- Opening a forty-file pull request and reading the approval as a verdict on the code.
- Spending review attention on formatting that a tool in the pipeline should decide.
- Committing intermediate states that do not build, which makes the history impossible to bisect.
- Adding a feature flag with no plan to remove it, until nobody knows which paths are live.
- Pushing a release commit and forgetting the tag, since tags do not travel with a normal push.
- Using a lightweight tag for a release, so there is no record of who tagged it or when.
- Publishing a changelog of commit summaries as though it were release notes, leaving consumers to work out whether the upgrade affects them.

23. Quick reference

Term	Meaning
Blob, tree, commit, tag	File contents, directory listing, snapshot with parents, named pointer
Ref	A name identifying an object, most often a commit. A branch is a ref pointing at a commit
Content-addressed	Every object stored under a hash of its own contents
Commit	A snapshot of a whole tree, not a difference. Diffs are computed on request
Branch	A movable pointer to one commit. A file holding a hash
HEAD	Where we are now. Usually a symbolic ref to a branch
Detached HEAD	HEAD points straight at a commit. Nothing is broken; name it before leaving
<code>origin/main</code>	Our last observed position of the remote branch. Stale until <code>git fetch</code>
Index, staging area	The proposed next commit. What makes a commit a selection rather than a snapshot of disk
<code>git fetch</code> against <code>git pull</code>	Pull is fetch plus an integration step. Fetch is always safe; pull changes the branch we are on
Ahead, behind, diverged	Our branch relative to its remote-tracking ref. Only meaningful after a fetch
Fast-forward	The target is a descendant, so the pointer moves and no merge commit is made
<code>--ff-only</code>	Allow only a fast-forward or nothing to do. Refuse when integrating would need a merge

Term	Meaning
Merge base	The most recent commit two branches share. What a three-way merge compares against
Rebase	Replays commits as new commits with new hashes
<code>--force-with-lease</code>	Refuses the push when the remote branch differs from the expected remote state. Prefer over <code>--force</code>
Reflog	Local record of every position HEAD and branch tips have held. Not pushed
Bisect	Binary search over history. Needs most commits to build. <code>git bisect skip</code> handles the rest
Hooks, CI, protection	Convenience, verification, enforcement. Only the last one refuses a merge
CODEOWNERS	Maps paths to required reviewers. Routes automatically, and becomes a queue if paths are too broad
<code>rerere</code>	Records a conflict resolution and reapplies it next time
Atomic commit	One logical change, leaving the code working
Conventional Commits	<code>type(scope): description</code> , enabling generated versions and changelogs
Trunk-based development	One branch, branches measured in hours, incomplete work behind flags
Branch by abstraction	Introduce an interface over current behaviour, move implementations behind it
Expand and contract	Add, write to both, migrate readers, remove. Each step deployable alone
Semantic versioning	<code>MAJOR.MINOR.PATCH</code> . Describes the interface, not the risk
Hyrum's Law	With enough consumers, every observable behaviour becomes something somebody relies on
Annotated against lightweight tag	With author, date and message, against a bare pointer. Use annotated for releases

Task	Command
See what the remote has, changing nothing local	<code>git fetch</code>
Integrate only if no merge is needed	<code>git pull --ff-only</code>
Check whether we are ahead, behind or diverged	<code>git fetch</code> then <code>git status</code>
Stage selected changes, hunk by hunk	<code>git add -p</code>
Undo the last commit, keep changes staged	<code>git reset --soft HEAD~1</code>
Undo the last commit, keep changes unstaged	<code>git reset --mixed HEAD~1</code>
Undo a commit already on a shared branch	<code>git revert <commit></code>

Task	Command
Unstage a file	<code>git restore --staged <file></code>
Discard working-tree changes to a file	<code>git restore <file></code>
Reword, squash or reorder the last five commits	<code>git rebase -i HEAD~5</code>
Push a rewritten branch safely	<code>git push --force-with-lease</code>
Find where a branch used to point	<code>git reflog show <branch></code>
Recover a deleted branch	<code>git branch <name> <hash from reflog></code>
Set aside changes including untracked files	<code>git stash push -u -m "message"</code>
Work on a second branch without disturbing this one	<code>git worktree add ../hotfix main</code>
Take one commit onto another branch	<code>git cherry-pick <commit></code>
Search history for a bad commit, unattended	<code>git bisect start</code> then <code>git bisect run <cmd></code>
Exclude an untestable commit from a bisect	<code>git bisect skip</code>
Remember conflict resolutions	<code>git config --global rerere.enabled true</code>
Tag a release and publish the tag	<code>git tag -a v1.4.0 -m "..."</code> then <code>git push origin v1.4.0</code>
Name a commit relative to the last tag	<code>git describe --tags</code>

Situation	What to reach for
Committed to the wrong branch	<code>git reset --soft HEAD~1</code> , switch, commit again
Committed something secret, not yet pushed	Amend or rebase it away, then force-with-lease if the branch was pushed earlier
Committed something secret and pushed it	Rotate the credential first. Rewriting removes references, not the exposure. Cleanup comes after
A merge commit appeared that nobody wanted	A <code>pull</code> on a diverged branch. Use <code>--ff-only</code> or <code>--rebase</code> deliberately
Last commit needs one more file	<code>git add <file></code> then <code>git commit --amend</code>
Branch has drifted far from the mainline	Rebase onto it now, not at the end. Conflicts scale with divergence
The same conflict keeps reappearing in a rebase	<code>rerere.enabled</code>
Something broke and nobody knows when	<code>git bisect run</code> with a failing test

Situation	What to reach for
Lost commits after a bad rebase or reset	<code>git reflog</code> , then give the commit we want a branch name
A fix is needed on a release branch only	Cherry-pick. This is the case it is for
The feature is not finished but the branch is old	Merge it behind a flag rather than letting it age